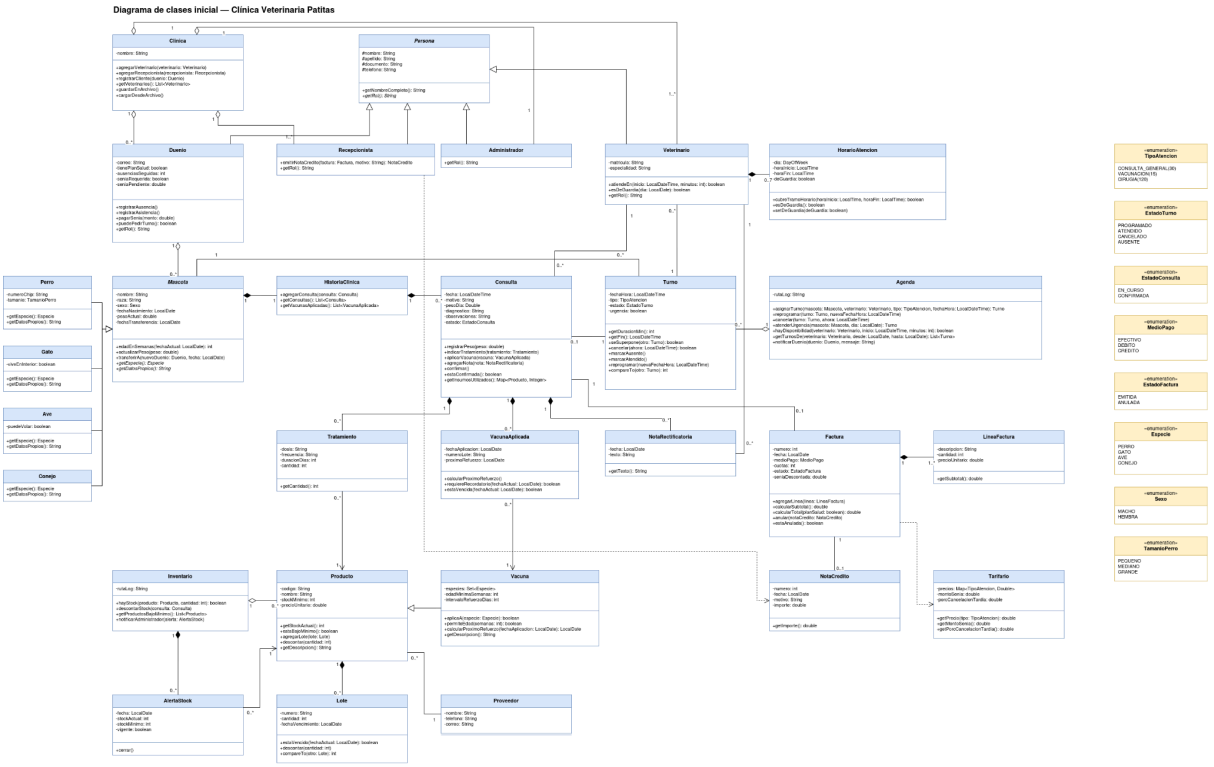
Punto 3 - Diagrama de clases inicial



Convenciones

- **Visibilidad:** + público, - privado, # protegido.
- **Clase abstracta:** su nombre y sus métodos abstractos van en cursiva en el diagrama.
- **Composición (rombo lleno):** la parte no existe sin el todo.
- **Agregación (rombo hueco):** el todo agrupa partes que existen por sí solas.
- **Asociación:** línea simple.
- Los atributos que son colecciones o referencias a otras clases del modelo no se repiten como atributos: quedan implícitos en la relación y su multiplicidad.
- Las enumeraciones se muestran aparte y se usan como tipos de atributos.

1. Clínica y personas

Clinica

Representa a la clínica «Patitas» y agrupa a su personal y a sus clientes.

- **Atributos:** -nombre: String.
- **Métodos:**

    - agregarVeterinario(veterinario), agregarRecepcionista(recepcionista) y registrarCliente(duenio): dan de alta cada elemento en la clínica.
    - getVeterinarios(): List<Veterinario>: expone la lista, por ejemplo para elegir veterinario al asignar un turno, para el reporte de turnos por veterinario o para buscar al de guardia.
    - guardarEnArchivo() y cargarDesdeArchivo(): guardan y recuperan los datos de la clínica en archivos para que se conserven entre ejecuciones.
- **Relaciones (todas agregaciones, con Clinica en el extremo del rombo):** 1 a 1..* con Veterinario, 1 a 1..* con Recepcionista, 1 a 1 con Administrador y 1 a 0..* con Duenio (clientes).

Persona (abstracta)

Generaliza los datos comunes de todas las personas del sistema.

- **Atributos (protegidos, para que las subclases los usen):** #nombre, #apellido, #documento, #telefono, todos String.
- **Métodos:**
    - getNombreCompleto(): String: une nombre y apellido; lo heredan todas las subclases.
    - getRol(): String (abstracto): cada subclase lo sobreescribe e informa qué tipo de persona es.
- **Relaciones:** es la superclase de Duenio, Veterinario, Recepcionista y Administrador.

Duenio

Cliente de la clínica, responsable de una o más mascotas.

- **Atributos:**
    - -correo: String.
    - -tienePlanSalud: boolean: da derecho al 15 % de descuento (RN-F3).
    - -ausenciasSeguidas: int: contador de ausencias; vuelve a cero cuando asiste a un turno (S-05).
    - -seniaRequerida: boolean: se activa con la tercera ausencia seguida (RN-T4).
    - -seniaPendiente: double: monto de seña a descontar de la próxima factura (S-04).
- **Métodos:**
    - registrarAusencia(): suma una ausencia y, al llegar a 3, exige seña.
    - registrarAsistencia(): reinicia el contador de ausencias.
    - pagarSenia(monto): registra el pago de la seña y lo habilita a pedir turnos.
    - puedePedirTurno(): boolean: false si tiene seña requerida sin pagar. Lo consulta Agenda al asignar un turno.
    - getRol(): sobreescribe el de Persona.
- **Relaciones:** hereda de Persona; agregación 1 a 0..* con Mascota (es el dueño actual); forma parte de Clinica como cliente.

Recepcionista

Personal que registra dueños y mascotas, gestiona turnos y factura.

- **Atributos:** ninguno propio.
- **Métodos:**
    - emitirNotaCredito(factura, motivo): NotaCredito: crea la nota de crédito por el total de la factura y la anula (CU-10, S-27).
    - getRol().
- **Relaciones:** hereda de Persona; asociación con NotaCredito (una recepcionista emite 0..* notas; cada nota la emite una recepcionista); parte de Clinica.

Administrador

Gestiona veterinarios, horarios, catálogos y consulta reportes. Recibe las alertas de stock bajo.

- **Atributos:** ninguno propio.
- **Métodos:** getRol().
- **Relaciones:** hereda de Persona; parte de Clinica (una clínica tiene un administrador).

Veterinario

Profesional que atiende, registra consultas, diagnósticos, tratamientos y vacunas, y firma notas rectificatorias (RN-M3, RN-M4).

- **Atributos:** -matricula: String, -especialidad: String. Sus horarios y guardias no son atributos, sino que se expresan en la relación con HorarioAtencion.
- **Métodos:**
    - atiendeEn(inicio, minutos): boolean: indica si el veterinario atiende en ese tramo; busca el HorarioAtencion del día y le pregunta si lo cubre (RN-T2, CU-11).
    - esDeGuardia(dia): boolean: indica si está de guardia ese día (S-07), según su HorarioAtencion.
    - getRol().
- **Relaciones:** hereda de Persona; composición 1 a 0..7 con HorarioAtencion; asociación 0..* a 1 con Turno, con Consulta y con NotaRectificatoria; parte de Clinica.

HorarioAtencion

Bloque de atención de un veterinario para un día de la semana. Hay como máximo un bloque por día (S-22).

- **Atributos:** -dia: DayOfWeek, -horaInicio: LocalTime, -horaFin: LocalTime, -deGuardia: boolean (si ese día el veterinario es el de guardia).
- **Métodos:**
    - cubreTramoHorario(horaInicio, horaFin): boolean: true si el tramo entra completo dentro del bloque (this.horaInicio <= horaInicio y horaFin <= this.horaFin).

    - esDeGuardia(): boolean y setDeGuardia(deGuardia): consulta y cambia la marca de guardia.
- **Relaciones:** composición con Veterinario: no existe sin él.

2. Mascotas y especies

Mascota (abstracta)

Paciente de la clínica. Cada especie agrega datos propios y su propio calendario de vacunación (RN-M2).

- **Atributos:** -nombre, -raza (String), -sexo: Sexo, -fechaNacimiento: LocalDate, -pesoActual: double, -fechaTransferencia: LocalDate.
- **Métodos:**
    - edadEnSemanas(fechaActual): int: se usa para validar la edad mínima de una vacuna (RN-V3, S-14).
    - actualizarPeso(peso): el peso actual se actualiza con el peso del día de cada consulta (S-23).
    - transferirA(nuevoDuenio, fecha): cambia de dueño y guarda la fecha de transferencia; la historia clínica queda intacta (RN-M1, S-24).
    - getEspecie(): Especie y getDatosPropios(): String (abstractos): los sobreescribe cada especie.
- **Relaciones:** es la superclase de Perro, Gato, Ave y Conejo; asociada a 1 Duenio (el actual) y 0..* Turno; composición 1 a 1 con HistoriaClinica.

Perro, Gato, Ave y Conejo

Subclases concretas de Mascota. Todas sobreescriben getEspecie() (devuelve su valor de Especie) y getDatosPropios() (devuelve sus datos específicos como texto).

- **Perro:** -numeroChip: String, -tamanio: TamanioPerro.
- **Gato:** -viveEnInterior: boolean.
- **Ave:** -puedeVolar: boolean.
- **Conejo:** sin atributos extra (S-12); igual tiene su propio calendario de vacunación, que se obtiene filtrando el catálogo por especie (S-13).

HistoriaClinica

Conjunto de consultas de una mascota. Solo crece: nada se borra ni se modifica (RN-M4).

- **Atributos:** ninguno propio; las consultas son la relación.
- **Métodos:**
    - agregarConsulta(consulta): único modo de incorporar registros.
    - getConsultas(): List<Consulta>: devuelve la lista en orden cronológico, de solo lectura (CU-25).

    - getVacunasAplicadas(): List<VacunaAplicada>: reúne las vacunas de todas las consultas, para el calendario y los recordatorios.
- **Relaciones:** composición 1 a 1 con Mascota; composición 1 a 0..* con Consulta.

3. Turnos y agenda

Turno

Reserva de una mascota con un veterinario para una fecha y hora, según un tipo de atención. Puede ser de urgencia.

- **Atributos:** -fechaHora: LocalDateTime, -tipo: TipoAtencion (define la duración, RN-T1), -estado: EstadoTurno, -urgencia: boolean.
- **Métodos:**
    - getDuracionMin(): int: devuelve los minutos según el tipo (30, 15 o 120).
    - getFin(): LocalDateTime: fechaHora más la duración.
    - seSuperpone(otro): boolean: detecta superposición con otro turno (RN-T2).
    - cancelar(ahora): boolean: cancela el turno; devuelve true si hubo cargo por cancelación tardía, es decir, si faltaban menos de 2 horas (RN-T3).
    - marcarAusente() y marcarAtendido(): cambian el estado.
    - reprogramar(nuevaFechaHora): cambia la fecha y hora de un turno programado.
    - compareTo(otro): int: orden natural por fecha y hora (Comparable<Turno>).
- **Relaciones:** agregado por Agenda (1 a 0..*); asociación 0..* a 1 con Mascota y con Veterinario; asociación 1 a 0..1 con Consulta (un turno atendido origina una consulta).

Agenda

Administra todos los turnos de la clínica y concentra las reglas de asignación.

- **Atributos:** -rutaLog: String, el archivo donde escribe las notificaciones.
- **Métodos:**
    - asignarTurno(mascota, veterinario, tipo, fechaHora): Turno: crea el turno si el dueño puede pedirlo y el veterinario está disponible (CU-04).
    - reprogramar(turno, nuevaFechaHora): valida disponibilidad y mueve el turno (CU-05).
    - cancelar(turno, ahora): cancela con o sin cargo (CU-06).
    - atenderUrgencia(mascota, dia): Turno: toma el primer horario del veterinario de guardia y desplaza el turno siguiente, en cascada si hace falta (CU-08, S-09).
    - hayDisponibilidad(veterinario, inicio, minutos): boolean: para validar que el turno está dentro del horario del vet y sin superposición (CU-11).
    - getTurnosDe(veterinario, desde, hasta): List<Turno>: base del reporte de turnos por veterinario.
    - notificarDuenio(duenio, mensaje), escribe en el log el aviso al dueño por reprogramaciones y desplazamientos por urgencia (CU-05, CU-08, S-02).
- **Relaciones:** agregación 1 a 0..* con Turno.

4. Atención clínica

Consulta

Registro de una atención. Tiene dos estados: en curso y confirmada (S-17). Al confirmarse queda inmodificable y habilita la facturación (S-18).

- **Atributos:** -fecha: LocalDateTime, -motivo, -diagnostico, -observaciones (String), -pesoDia: Double (puede ser nulo si no se registró), -estado: EstadoConsulta.
- **Métodos:**
  - registrarPeso(peso): guarda el peso del día; sin él no se puede indicar medicación (RN-M5).
  - indicarTratamiento(tratamiento): lo agrega si hay peso y stock (CU-17).
  - aplicarVacuna(vacuna): agrega la vacuna aplicada (CU-18).
  - agregarNota(nota): asocia una nota rectificatoria (CU-24).
  - confirmar(): pasa a confirmada; el descuento de stock lo hace Inventario.
  - estaConfirmada(): boolean.
  - getInsumosUtilizados(): Map<Producto, Integer>: insumos con cantidad, para descontar stock y armar las líneas de la factura.
- **Relaciones:** composición con HistoriaClinica; composición 1 a 0..* con Tratamiento, VacunaAplicada y NotaRectificatoria; asociación a 1 Veterinario y 0..1 Turno; asociada 0..1 a Factura.

Tratamiento

Indicación de un medicamento dentro de una consulta.

- **Atributos:** -dosis: String, -frecuencia: String, -duracionDias: int, -cantidad: int (unidades que se consumirán al confirmar).
- **Métodos:** getCantidad(): int.
- **Relaciones:** composición con Consulta; asociación hacia Producto (el medicamento).

VacunaAplicada

Cada vacuna que se aplicó a una mascota, con su lote y su próximo refuerzo (RN-V1).

- **Atributos:** -fechaAplicacion: LocalDate, -numeroLote: String, -proximoRefuerzo: LocalDate.
- **Métodos:**
  - calcularProximoRefuerzo(): suma el intervalo de la vacuna a la fecha de aplicación (S-15).
  - requiereRecordatorio(fechaActual): boolean: true desde 7 días antes del refuerzo (RN-V2).
  - estaVencida(fechaActual): boolean: true si pasaron 30 días del refuerzo sin aplicarlo (RN-V2, S-26).
- **Relaciones:** composición con Consulta; asociación hacia Vacuna.

NotaRectificatoria

Corrige un error de una consulta sin modificarla (RN-M4, S-21).

- **Atributos:** -fecha: LocalDate, -texto: String.
- **Métodos:** getTexto(): String.
- **Relaciones:** composición con Consulta; asociación 0..* a 1 con Veterinario (quien la firma).

5. Inventario y stock

Inventario

Controla el stock de medicamentos, insumos y vacunas, y concentra el descuento al confirmar una consulta.

- **Atributos:** -rutaLog: String, el archivo donde escribe las alertas.
- **Métodos:**
  - hayStock(producto, cantidad): boolean: verifica que haya existencias en lotes no vencidos (RN-S2); se usa al indicar un tratamiento, sin descontar todavía.
  - descontarStock(consulta): al confirmar la consulta, descuenta cada insumo siguiendo FEFO, es decir, del lote que vence antes (RN-S3, CU-22). Si el stock total no alcanza, no descuenta nada. Si un producto queda en su mínimo o por debajo, genera una AlertaStock sin duplicar una vigente (RN-S1, CU-23).
  - getProductosBajoMinimo(): List<Producto>: base del reporte de productos con stock bajo.
  - notificarAdministrador(alerta: AlertaStock), escribe la alerta de stock bajo en el log (CU-23).
- **Relaciones:** agregación 1 a 0..* con Producto; composición 1 a 0..* con AlertaStock.

Producto

Cualquier insumo o medicamento que la clínica compra y vende.

- **Atributos:** -codigo: String (único), -nombre: String, -stockMinimo: int, -precioUnitario: double (precio de venta, S-11).
- **Métodos:**
  - getStockActual(): int: suma las cantidades de sus lotes no vencidos (S-16).
  - estaBajoMinimo(): boolean: true si el stock actual es menor o igual al mínimo.
  - agregarLote(lote): registra un lote recibido; rechaza uno ya vencido.
  - descontar(cantidad): toma la cantidad de los lotes ordenados por vencimiento (FEFO).
  - getDescripcion(): String: texto descriptivo; lo sobreescribe Vacuna.
- **Relaciones:** composición 1 a 0..* con Lote; asociación 0..* a 1 con Proveedor; agregado por Inventario; superclase de Vacuna; referenciado por Tratamiento y por AlertaStock.

Vacuna

Entrada del catálogo de vacunas. Es un producto más (S-25): tiene lotes, vencimiento físico y precio.

- **Atributos:** -especies: Set<Especie> (a qué especies se aplica), -edadMinimaSemanas: int (S-14), -intervaloRefuerzoDias: int (S-15).
- **Métodos:**
  - aplicaA(especie): boolean: sirve para armar el calendario de cada especie (S-13).
  - permiteEdad(semanas): boolean: valida la edad mínima (RN-V3).
  - calcularProximoRefuerzo(fechaAplicacion): LocalDate: fecha de aplicación más el intervalo.
  - getDescripcion(): sobreescribe el de Producto.
- **Relaciones:** hereda de Producto; referenciada por VacunaAplicada.

Lote

Partida de un producto con su propia fecha de vencimiento.

- **Atributos:** -numero: String, -cantidad: int, -fechaVencimiento: LocalDate (vencimiento físico, independiente de la vigencia de la inmunidad, S-26).
- **Métodos:**
  - estaVencido(fechaActual): boolean: un lote vencido no puede usarse (RN-S3).
  - descontar(cantidad): resta unidades del lote.
  - compareTo(otro): int: orden natural por vencimiento, que implementa FEFO (Comparable<Lote>).
- **Relaciones:** composición con Producto.

Proveedor

Empresa que abastece productos (S-28).

- **Atributos:** -nombre, -telefono, -correo (String).
- **Métodos:** ninguno propio.
- **Relaciones:** un proveedor abastece 0..* productos y cada producto tiene un único proveedor.

AlertaStock

Aviso de reposición que recibe el administrador (RN-S1).

- **Atributos:** -fecha: LocalDate, -stockActual: int, -stockMinimo: int, -vigente: boolean (evita duplicar alertas del mismo producto).
- **Métodos:** cerrar(): la marca como no vigente cuando se repone el stock.
- **Relaciones:** composición con Inventario; asociación 0..* a 1 con Producto.

6. Facturación

Factura

Comprobante de una consulta confirmada: incluye la consulta y los insumos usados (RN-F1). Una vez emitida no se modifica (RN-F4).

- **Atributos:**
  - -numero: int, -fecha: LocalDate.
  - -medioPago: MedioPago, -cuotas: int (para crédito).
  - -estado: EstadoFactura (emitida o anulada).
  - -seniaDescontada: double: parte de la seña pendiente del dueño que se descontó del total (S-04). Queda registrada porque la factura no se puede modificar después.
- **Métodos:**
  - agregarLinea(linea): agrega una línea de detalle.
  - calcularSubtotal(): double: suma los subtotales de las líneas.
  - calcularTotal(planSalud): double: aplica descuentos y recargos según el medio de pago y el plan de salud (RN-F2, RN-F3, S-19) y resta la seña.
  - anular(notaCredito): verifica que no esté anulada y cambia el estado a anulada, sin tocar su contenido (CU-10).
  - estaAnulada(): boolean.
- **Relaciones:** composición 1 a 1..* con LineaFactura; asociación con Consulta (una consulta confirmada tiene 0..1 factura); dependencia hacia Tarifario (precios); referenciada por NotaCredito.

LineaFactura

Un renglón del detalle: la consulta o un insumo.

- **Atributos:** -descripcion: String, -cantidad: int, -precioUnitario: double.
- **Métodos:** getSubtotal(): double (cantidad × precioUnitario).
- **Relaciones:** composición con Factura.

NotaCredito

Anula una factura completa; no hay anulaciones parciales (S-20).

- **Atributos:** -numero: int, -fecha: LocalDate, -motivo: String, -importe: double (igual al total de la factura).
- **Métodos:** getImporte(): double.
- **Relaciones:** asociación 0..1 a 1 con Factura; asociación 0..* a 1 con Recepcionista (quien la emite, S-27).

Tarifario

Concentra los valores configurables del negocio (S-03, S-10).

- **Atributos:** -precios: Map<TipoAtencion, Double> (consulta general, vacunación y cirugía), -montoSenia: double (monto fijo), -porcCancelacionTardia: double (20 %, S-06).
- **Métodos:** getPrecio(tipo): double, getMontoSenia(): double y getPorcCancelacionTardia(): double.
- **Relaciones:** Factura depende de él para obtener el precio de la consulta y calcular el cargo por cancelación tardía.

7. Enumeraciones

Enumeración	Valores	Se usa en
TipoAtencion	CONSULTA_GENERAL(30), VACUNACION(15), CIRUGIA(120); entre paréntesis, la duración en minutos	Turno, Tarifario
EstadoTurno	PROGRAMADO, ATENDIDO, CANCELADO, AUSENTE	Turno
EstadoConsulta	EN_CURSO, CONFIRMADA	Consulta
MedioPago	EFECTIVO, DEBITO, CREDITO	Factura
EstadoFactura	EMITIDA, ANULADA	Factura
Especie	PERRO, GATO, AVE, CONEJO	Mascota.getEspecie(), Vacuna
Sexo	MACHO, HEMBRA	Mascota
TamanioPerro	PEQUENO, MEDIANO, GRANDE	Perro